

NumPy — Nền tảng tính toán cho AI

Thư viện cho Trí tuệ nhân tạo

Trình bày: Võ Luyện

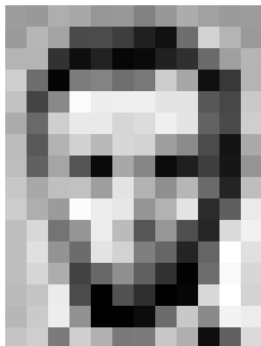
Nội dung buổi học

- 1 Vì sao AI cần NumPy?
- 2 Bắt đầu với NumPy
- 3 Các cách tạo mảng
- 4 Thuộc tính và kiểu dữ liệu
- 5 Truy cập phần tử: indexing và slicing
- 6 View và Copy — cái bẫy kinh điển
- 7 Vector hóa — tính toán không cần vòng lặp
- 8 Broadcasting
- 9 Hàm thống kê và khái niệm axis
- 10 Biến đổi hình dạng và ghép mảng
- 11 Đại số tuyến tính với NumPy
- 12 NumPy trong các bài toán AI
- 13 Bảng tra cứu
- 14 Thực hành

Sau buổi học, bạn sẽ làm được gì?

- Tạo và đọc được một ndarray
- Hiểu shape, ndim, dtype
- Truy cập dữ liệu: indexing, slicing, lọc theo điều kiện (masking)
- Viết phép tính trên cả mảng thay cho vòng lặp (vector hóa)
- Hiểu trục (axis) và chọn đúng trục khi tính toán
- Hiểu **broadcasting**
- Đổi hình dạng mảng và làm các phép ma trận cơ bản

Dữ liệu trong AI là những con số — rất nhiều con số



157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	105	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	105	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

- Máy tính **không "nhìn thấy" ảnh** — nó chỉ thấy một **ma trận các con số**, mỗi số là độ sáng của một điểm ảnh (pixel), từ 0 (đen) đến 255 (trắng)
- Chỉ một ảnh bé xíu như trên đã là hàng trăm con số. Một tấm ảnh 224×224 màu $\rightarrow 224 \times 224 \times 3 = 150\,528$ con số
- Bảng điểm 1000 học sinh $\times$ 10 môn $\rightarrow 10\,000$ con số

Vậy xử lý ngần ấy con số bằng gì?

Câu hỏi

Nếu dùng `list` của Python thuần và vòng lặp `for` để xử lý từng con số một, chuyện gì sẽ xảy ra?

Vậy xử lý ngàn ấy con số bằng gì?

Câu hỏi

Nếu dùng `list` của Python thuần và vòng lặp `for` để xử lý từng con số một, chuyện gì sẽ xảy ra?

Trả lời

Chương trình chạy **rất chậm**. Với hàng triệu con số, thứ đáng lẽ chạy trong vài giây có thể mất hàng phút, thậm chí hàng giờ.

Vậy xử lý ngàn ấy con số bằng gì?

Câu hỏi

Nếu dùng `list` của Python thuần và vòng lặp `for` để xử lý từng con số một, chuyện gì sẽ xảy ra?

Trả lời

Chương trình chạy **rất chậm**. Với hàng triệu con số, thứ đáng lẽ chạy trong vài giây có thể mất hàng phút, thậm chí hàng giờ.

Hướng giải quyết

Ta cần một công cụ chuyên để chứa và tính toán trên **cả một khối số cùng lúc** — đó chính là **NumPy**.

NumPy là gì?

- **NumPy** (Numerical Python) — thư viện tính toán số học nền tảng của Python
- Ra đời năm 2006, hiện là "trái tim" của toàn bộ hệ sinh thái khoa học dữ liệu và AI
- Cung cấp kiểu dữ liệu **ndarray** — mảng nhiều chiều, lưu số hiệu quả
- Phần lõi viết bằng **C** và Fortran → tính toán trên mảng lớn nhanh hơn đáng kể so với vòng lặp Python

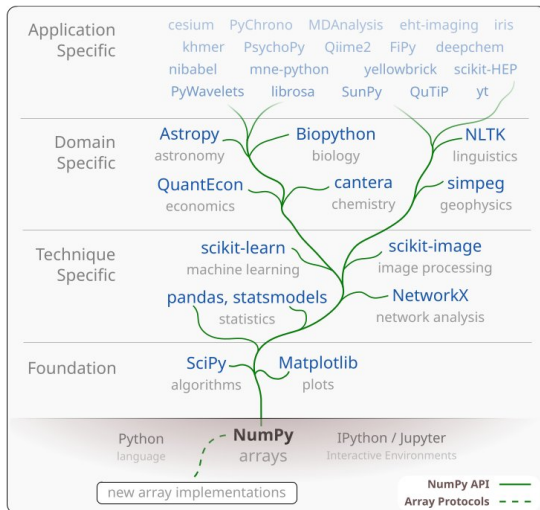
NumPy là gì?

- **NumPy** (Numerical Python) — thư viện tính toán số học nền tảng của Python
- Ra đời năm 2006, hiện là "trái tim" của toàn bộ hệ sinh thái khoa học dữ liệu và AI
- Cung cấp kiểu dữ liệu **ndarray** — mảng nhiều chiều, lưu số hiệu quả
- Phần lõi viết bằng **C** và Fortran → tính toán trên mảng lớn nhanh hơn đáng kể so với vòng lặp Python

Vị trí của NumPy

- NumPy là **nền tảng** của hệ sinh thái khoa học dữ liệu Python
- **Pandas, Scikit-learn, Matplotlib, OpenCV** tương tác chặt chẽ với ndarray
- **PyTorch, TensorFlow** có kiểu **Tensor** riêng, nhưng **chuyển đổi qua lại** được với mảng NumPy

Vị trí của NumPy trong hệ sinh thái AI



Nguồn: Harris et al., *Array programming with NumPy*, Nature 585 (2020)

So sánh tốc độ: list vs ndarray

```
import numpy as np, time

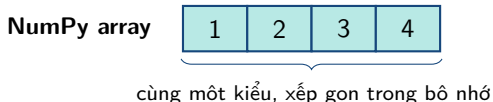
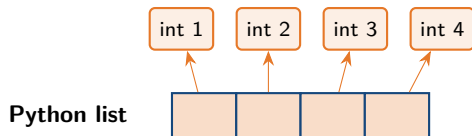
n = 1_000_000
list_a = list(range(n))
arr_a = np.arange(n)

t0 = time.time()
tong1 = sum(x * x for x in list_a)    # pure Python
t1 = time.time()
tong2 = np.sum(arr_a * arr_a)         # NumPy
t2 = time.time()
print("Python:", t1 - t0, " NumPy:", t2 - t1)
```

Nhận xét

Với phép tính trên mảng lớn, NumPy **thường nhanh hơn đáng kể** so với vòng lặp Python. Mức chênh lệch phụ thuộc bài toán và phần cứng — hãy tự đo trên máy mình.

Vì sao NumPy nhanh hơn?



Mỗi ô chỉ là **con trỏ** tới một đối tượng Python nằm rải rác trong bộ nhớ. Mỗi phép cộng phải: tìm đối tượng → kiểm tra kiểu → tính → tạo đối tượng mới.

Các số **cùng kiểu**, được tổ chức gọn trong bộ nhớ → vòng lặp chạy trong C, tận dụng cache CPU và lệnh xử lý song song (SIMD).

NumPy array khác Python list ở điểm nào?

	Python list	NumPy ndarray
Kiểu phần tử	Trộn lẫn tùy ý	Đồng nhất (int, float, ...)
Kích thước	Thay đổi được	Cố định khi tạo
Bộ nhớ	Nhiều, rải rác	Gọn, tổ chức hiệu quả
Tốc độ tính toán	Chậm (vòng lặp)	Nhanh (chạy trong C)
Phép toán số học	$[1, 2] + [3, 4] \rightarrow$ nối	$a + b \rightarrow$ cộng từng phần tử
Nhiều chiều	Phải lồng list	Hỗ trợ sẵn
Hàm toán học	Phải tự viết	Có sẵn hàng trăm hàm

Tóm lại

List dùng để chứa **mọi thứ**. NumPy array dùng để **tính toán trên số**.

Cài đặt và import

Cài đặt (chỉ làm một lần, trong Terminal / Command Prompt):

```
pip install numpy
```

Với Google Colab hoặc Anaconda: NumPy đã được cài sẵn. **Import** — quy ước chung của cả thế giới:

```
import numpy as np  
  
print(np.__version__)  # check the installed version
```

as np nghĩa là gì?

as đặt cho thư viện một **tên gọi tắt** (tiếng Anh: *alias*). Sau dòng đó, thay vì viết `numpy.array(...)` ta chỉ cần viết `np.array(...)` — ngắn hơn, gõ nhanh hơn.

Mảng đầu tiên của bạn

```
import numpy as np

a = np.array([1, 2, 3, 4, 5])
print(a)           # [1 2 3 4 5]
print(type(a))     # <class 'numpy.ndarray'>

# one operation applies to the WHOLE array
print(a + 10)      # [11 12 13 14 15]
print(a * 2)       # [ 2  4  6  8 10]
print(a ** 2)      # [ 1  4  9 16 25]
```

- Không cần vòng lặp `for` — đây gọi là **vector hóa** (vectorization)
- So sánh: với list, `[1,2,3] * 2` cho `[1,2,3,1,2,3]` — hoàn toàn khác!

Mảng nhiều chiều — khái niệm cốt lõi

3	1	4	1
---	---	---	---

1 chiều (1D)

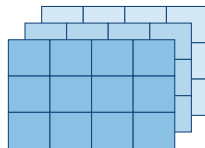
vector — shape (4,)

vd: điểm 4 môn học

2 chiều (2D)

ma trận — shape (3,4)

vd: bảng điểm 3 HS $\times$ 4 môn



3 chiều (3D)

tensor — shape (3,4,3)

vd: ảnh màu cao $\times$ rộng $\times$ RGB

Thuật ngữ

scalar (số đơn lẻ) $\rightarrow$ 0 chiều | **vector** $\rightarrow$ 1 chiều | **matrix** (ma trận) $\rightarrow$ 2 chiều. **Tensor** là cách gọi tổng quát cho dữ liệu nhiều chiều. Trong NumPy, kiểu dữ liệu cụ thể ta đang học là **ndarray**.

Cách đọc shape

shape	ndim	Ý nghĩa
(4,)	1	1 trục, kích thước 4
(3, 4)	2	trục 0 kích thước 3, trục 1 kích thước 4
(2, 3, 4)	3	trục 0 là 2, trục 1 là 3, trục 2 là 4

Quy tắc

Số lượng con số trong **shape** = số chiều **ndim**. Mỗi con số = kích thước của một **trục** (axis).

```
a = np.zeros((3, 4))  
a.shape    # (3, 4)    -> two numbers  
a.ndim     # 2         -> two axes: axis 0 and axis 1
```

Tạo mảng 2 chiều

```
diem = np.array([[8.0, 7.5, 9.0],      # student 1
                 [6.5, 8.0, 7.0],      # student 2
                 [9.0, 9.5, 8.5]])      # student 3

print(diem)
print(diem.shape)    # (3, 3)  -> 3 rows, 3 columns
print(diem.ndim)     # 2       -> number of dimensions
```

- Mảng 2D được tạo từ **list của các list**
- Mỗi list con là một **hàng** (row) và phải có **cùng độ dài**
- Quy ước đọc shape: (số hàng, số cột)

Tạo mảng từ giá trị có sẵn

```
np.zeros(shape, dtype)    # SYNTAX: shape first, then dtype

np.zeros(5)                # shape = 5          -> [0. 0. 0. 0. 0.]
np.zeros((2, 3))           # shape = (2, 3)     -> 2x3 matrix of 0
np.ones((2, 3))            # 2x3 matrix of 1
np.full((2, 3), 7)         # (shape, value)     -> 2x3 matrix of 7
np.eye(3)                  # 3x3 identity matrix (diagonal = 1)
```

- Tham số **thứ nhất** là **shape** — hình dạng. Nhiều chiều thì phải gói tất cả vào **một** cặp ngoặc: `(2, 3)`. Đó là lý do `np.zeros(2, 3)` có **hai** lớp ngoặc
- Tham số **thứ hai** là **dtype** — kiểu dữ liệu, **không phải** số cột

Vì vậy `np.zeros(2, 3)` sẽ báo lỗi

NumPy hiểu thành `shape = 2, dtype = 3` — mà 3 không phải là một kiểu dữ liệu.

Tạo dãy số: arange và linspace

```
np.arange(start, stop, step)    # count from start, add step
np.linspace(start, stop, num)   # split [start, stop] evenly

np.arange(5)                    # [0 1 2 3 4]           stop = 5
np.arange(2, 10)                # [2 3 4 5 6 7 8 9]       start=2, stop=10
np.arange(0, 10, 2)             # [0 2 4 6 8]           step = 2
np.linspace(0, 1, 5)            # [0. 0.25 0.5 0.75 1.]   num = 5
```

Tham số	Ý nghĩa
start	giá trị bắt đầu — luôn được lấy
stop	giá trị dừng: arange không lấy, linspace có lấy
step	bước nhảy — chỉ arange có
num	số phần tử — chỉ linspace có

Tạo mảng ngẫu nhiên

```
rng = np.random.default_rng(42)    # 42 is the seed

rng.random(5)                       # 5 random floats in [0, 1)
rng.random((2, 3))                  # 2x3 matrix of random floats
rng.integers(1, 7, 10)              # 10 integers in 1..6 (dice)
rng.normal(0, 1, 5)                 # 5 values, normal(mean 0, std 1)
```

Vai trò của seed

Đặt seed → mọi lần chạy đều cho **cùng một kết quả** → thí nghiệm **lặp lại được** (reproducible).

Các thuộc tính quan trọng của mảng

```
a = np.array([[1, 2, 3],  
             [4, 5, 6]])
```

Thuộc tính	Ý nghĩa	Với a ở trên
a.shape	hình dạng: (số hàng, số cột)	(2, 3)
a.ndim	số chiều	2
a.size	tổng số phần tử	6
a.dtype	kiểu dữ liệu của phần tử	int64
a.itemsize	số byte của mỗi phần tử	8
a.nbytes	tổng số byte (size × itemsize)	48

- Đây là **thuộc tính**, không phải hàm → viết `a.shape`, **không** có dấu ngoặc

Kiểu dữ liệu (dtype)

```
np.array([1, 2, 3]).dtype           # int64 (old Windows: int32)
np.array([1.0, 2.0]).dtype          # float64
np.array([1, 2], dtype=np.float32) # set dtype at creation

a = np.array([1.7, 2.3, 3.9])
b = a.astype(int)                   # [1 2 3] - TRUNCATES the decimals
c = a.astype(np.float32)
```

Kiểu	Bộ nhớ	Dùng khi
uint8	1 byte	Ảnh (giá trị 0–255)
int32/int64	4/8 byte	Nhãn, chỉ số, số đếm
float32	4 byte	Số thực, tiết kiệm bộ nhớ
float64	8 byte	Mặc định của NumPy, độ chính xác cao
bool	1 byte	Mặt nạ lọc dữ liệu

Cẩn thận với kiểu số nguyên

```
a = np.array([250, 251, 252], dtype=np.uint8)
print(a + 10)          # [4 5 6] ??? -- OVERFLOW!
# uint8 only holds 0..255, so 260 wraps around to 4

b = np.array([1, 2, 3])
b[0] = 3.9
print(b)               # [3 2 3] -- truncated to integer
```

- Cần tính toán trên số nguyên nhỏ → ép sang số thực trước:
`a.astype(np.float32)`

Indexing và slicing mảng 1 chiều

```
a = np.array([10, 20, 30, 40, 50, 60])

a[0]          # 10    first element
a[-1]         # 60    last element
a[1:4]        # [20 30 40]    index 1 up to 3
a[:3]         # [10 20 30]    first 3 elements
a[3:]         # [40 50 60]    from index 3 to the end
a[::2]        # [10 30 50]    every other element
a[::-1]       # [60 50 40 30 20 10]    reversed
```

- Cú pháp giống hệt list của Python: `a[start : stop : step]`
— bắt đầu, kết thúc, bước nhảy
- Luôn nhớ: chỉ số bắt đầu từ **0**, và **không** lấy phần tử ở vị trí `stop`

Indexing mảng 2 chiều

```
a = np.array([[ 1,  2,  3,  4],
               [ 5,  6,  7,  8],
               [ 9, 10, 11, 12]])

a[1, 2]      # 7      row 1, column 2
a[1][2]      # 7      list style - slower, AVOID
a[0]         # [1 2 3 4] the whole row 0
a[:, 1]      # [2 6 10] the whole column 1
a[0:2, 1:3]  # [[2 3], [6 7]] 2x2 sub-array
a[-1, -1]   # 12      bottom-right element
```

Cách đọc

Dấu phẩy ngăn cách các chiều: `a[i, j]` — hàng `i`, cột `j`. Dấu hai chấm : nghĩa là "lấy tất cả".

Minh họa slicing 2 chiều

1	2	3	4
5	6	7	8
9	10	11	12

`a[0]`

1	2	3	4
5	6	7	8
9	10	11	12

`a[:, 1]`

1	2	3	4
5	6	7	8
9	10	11	12

`a[0:2, 1:3]`

Lọc dữ liệu bằng điều kiện (boolean masking)

```
diem = np.array([8.5, 4.0, 7.0, 9.5, 5.5, 3.0])

mask = diem >= 5
print(mask)           # [True False True True True False]
print(diem[mask])     # [8.5 7.  9.5 5.5] keeps scores >= 5
print(diem[diem >= 5]) # short form, very common

print(np.sum(diem >= 5)) # 4 - count of passes
print(np.mean(diem >= 5)) # 0.666... - pass rate

# combine conditions with & (and), | (or), ~ (not)
print(diem[(diem >= 5) & (diem < 9)]) # [8.5 7.  5.5]
```

Lưu ý

Dùng **&**, **|** chứ **không** dùng **and**, **or**. Và **bắt buộc** có dấu ngoặc quanh mỗi điều kiện.

Fancy indexing và np.where

Lấy nhiều phần tử theo danh sách chỉ số:

```
a = np.array([10, 20, 30, 40, 50])
a[[0, 2, 4]]           # [10 30 50]
a[[4, 4, 0]]           # [50 50 10] - repeats, any order
```

np.where — "nếu ...thì ...ngược lại":

```
diem = np.array([8.5, 4.0, 7.0, 3.0])

np.where(diem >= 5, 1, 0)      # [1 0 1 0] - label pass/fail
np.where(diem >= 5, diem, 0)  # [8.5 0. 7. 0.] - fail -> 0
np.where(diem >= 5)           # (array([0, 2]),) - positions
```

- np.where chính là cách viết if-else cho cả mảng, không cần vòng lặp

Slicing tạo ra "view", không phải bản sao

```
a = np.array([1, 2, 3, 4, 5])
b = a[1:4]          # b is a VIEW - same memory as a
b[0] = 99

print(b)            # [99  3  4]
print(a)            # [ 1 99  3  4  5]  <-- a CHANGED TOO!
```

Slicing tạo ra "view", không phải bản sao

```
a = np.array([1, 2, 3, 4, 5])
b = a[1:4]          # b is a VIEW - same memory as a
b[0] = 99

print(b)            # [99  3  4]
print(a)            # [ 1 99  3  4  5]  <-- a CHANGED TOO!
```

Muốn có bản sao độc lập → dùng `.copy()`:

```
c = a[1:4].copy()
c[0] = 0
print(a)            # [ 1 99  3  4  5]  <-- a unchanged
```

Vì sao NumPy làm vậy?

Để **tiết kiệm bộ nhớ** — cắt một mảng 1GB mà phải sao chép thì rất tốn kém.

Khi nào là view, khi nào là copy?

Thao tác	Kết quả
<code>b = a[1:4]</code>	View — dùng chung bộ nhớ
<code>b = a[a > 5]</code>	Copy (boolean masking)
<code>b = a[[0, 2]]</code>	Copy (fancy indexing)
<code>b = a.reshape(2, 3)</code>	View (nếu có thể)
<code>b = a.T</code>	View
<code>b = a.copy()</code>	Copy
<code>b = a + 0</code>	Copy (mảng mới từ phép toán)

```
b = a[1:4]
print(b.base is a)    # True  -> b is a view of a
```

- Muốn sửa dữ liệu mà không ảnh hưởng mảng gốc → dùng `.copy()`

Phép toán trên từng phần tử (element-wise)

```
a = np.array([1, 2, 3, 4])
b = np.array([10, 20, 30, 40])

a + b      # [11 22 33 44]
b - a      # [ 9 18 27 36]
a * b      # [ 10 40 90 160]  ELEMENT-WISE product
b / a      # [10. 10. 10. 10.]
a ** 2     # [ 1  4  9 16]
b % 3      # [1 2 0 1]

a > 2      # [False False  True  True]
```

- * nhân **từng phần tử** và **có** broadcasting — không bắt buộc hai mảng cùng hình dạng (quy tắc ở phần sau). @ mới là phép nhân ma trận
- Kết quả luôn là một mảng **mới**

Các hàm toán học có sẵn (universal functions)

```
a = np.array([1, 4, 9, 16])

np.sqrt(a)          # [1.  2.  3.  4.]      square root
np.exp(a)           # e^a                  exponential
np.log(a)           # ln(a)               natural log
np.abs(-a)          # absolute value
np.round(np.array([1.234, 5.678]), 1)    # [1.2 5.7]

x = np.linspace(0, np.pi, 5)
np.sin(x)           # trigonometric functions
np.maximum(a, 5)     # [ 5  5  9 16]      element-wise max with 5
```

- Mọi hàm đều tác động lên **từng phần tử**, trả về mảng mới cùng hình dạng

Vector hóa: viết lại vòng lặp cho gọn và nhanh

Bài toán

Mảng `diem` chứa điểm của cả lớp. Cần quy đổi sang thang mới theo công thức **điểm mới** = **điểm cũ** $\times 2 + 1$ cho **mọi** học sinh.

Cách viết ngây thơ — duyệt từng học sinh một:

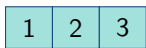
```
ket_qua = []                                # empty list
for i in range(len(diem)):                  # loop over students
    ket_qua.append(diem[i] * 2 + 1)         # compute, then append
ket_qua = np.array(ket_qua)                # back to a NumPy array
```

Cách viết NumPy — làm một lần cho cả mảng:

```
ket_qua = diem * 2 + 1
```

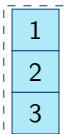
- Cùng kết quả, 1 dòng thay cho 4 dòng, và thường nhanh hơn đáng kể

Trước khi vào broadcasting: $(3,)$, $(3, 1)$ và $(1, 3)$



$(3,)$

1 trục — $\text{ndim} = 1$



$(3, 1)$

2 trục — 3 hàng $\times$ 1 cột



$(1, 3)$

2 trục — 1 hàng $\times$ 3 cột

```
v = np.array([1, 2, 3])  
v.shape          # (3,)      1 axis  
v.reshape(3, 1).shape # (3, 1)  2 axes  
v.reshape(1, 3).shape # (1, 3)  2 axes
```

- Ba mảng chứa cùng ba con số nhưng **shape khác nhau** $\rightarrow$ broadcasting xử lý khác nhau

Broadcasting — khi hai mảng khác hình dạng

```
a = np.array([1, 2, 3])
print(a + 10)          # [11 12 13] - 10 is "broadcast" to all

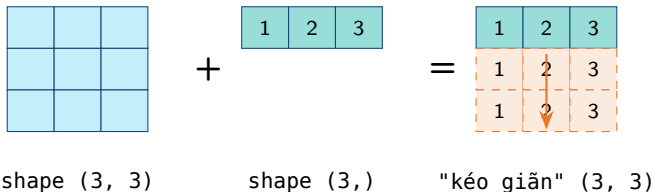
diem = np.array([[8.0, 7.0, 9.0],      # 3 students
                 [6.0, 8.5, 7.5],      # x 3 subjects
                 [9.0, 9.0, 8.0]])
he_so = np.array([2, 1, 1])           # weight per subject

print(diem * he_so)                  # every ROW is multiplied by he_so
```

Broadcasting là gì?

NumPy tự động "kéo giãn" mảng nhỏ hơn cho khớp hình dạng mảng lớn — **không tốn thêm bộ nhớ**, không cần vòng lặp.

Minh họa broadcasting



- Hàng [1 2 3] được lặp lại cho **mọi hàng** của ma trận
- Việc "kéo giãn" chỉ là **khái niệm** — NumPy không thực sự tạo bản sao trong bộ nhớ

Quy tắc broadcasting

NumPy so sánh kích thước từng **trục**, căn từ **phải sang trái**. Hai kích thước tương thích khi:

- ① chúng **bằng nhau**, hoặc
- ② một trong hai **bằng 1**

Nếu một mảng **thiếu trục** ở bên trái, ta tưởng tượng thêm số **1** vào bên trái.

	trục 0	trục 1
A — (3, 4)	3	4
B — (4,)	thiếu → 1	4
So sánh	3 với 1 → OK	4 với 4 → OK
Kết quả	3	4

Kết quả có shape = (3, 4)

Khi nào broadcasting thất bại?

	trục 0	trục 1
A — (3, 4)	3	4
B — (3,)	thiếu → 1	3
So sánh	3 với 1 → OK	4 với 3 → không khớp

- Hai mảng đều có số 3, nhưng vì căn **từ phải sang trái** nên số 3 của B bị đem so với số 4 của A → lỗi
- Muốn ghép theo **hàng**, phải đưa B về (3, 1):

```
A = np.zeros((3, 4))  
b = np.array([1, 2, 3])
```

```
A + b                # error: shapes (3,4) and (3,)  
A + b.reshape(3, 1) # OK: (3,4) with (3,1) -> (3,4)
```

Ứng dụng broadcasting: chuẩn hóa dữ liệu

Chuẩn hóa z-score: đưa mỗi cột về cùng một thang đo theo công thức

$$z = \frac{x - \mu}{\sigma}$$

với x là giá trị gốc, μ là trung bình của cột, σ là độ lệch chuẩn của cột.
Sau chuẩn hóa, mỗi cột có trung bình 0 và độ lệch chuẩn 1.

```
# X: 100 samples, each with 4 features
X = rng.normal(50, 10, size=(100, 4))

mean = X.mean(axis=0)    # shape (4,) - mean of EACH COLUMN
std  = X.std(axis=0)     # shape (4,) - std of each column

X_chuan = (X - mean) / std    # broadcast (100,4) with (4,)

print(X_chuan.mean(axis=0))  # ~ [0. 0. 0. 0.]
print(X_chuan.std(axis=0))   # ~ [1. 1. 1. 1.]
```

- Một dòng thay cho hai vòng lặp lồng nhau và 400 phép tính

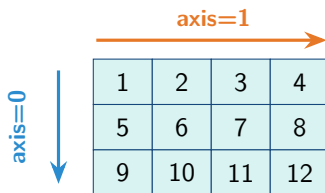
Các hàm tổng hợp

```
a = np.array([3, 1, 4, 1, 5, 9, 2, 6])
```

Hàm	Ý nghĩa	Với a ở trên
<code>a.sum()</code>	tổng các phần tử	31
<code>a.mean()</code>	trung bình cộng	3.875
<code>a.min()</code>	giá trị nhỏ nhất	1
<code>a.max()</code>	giá trị lớn nhất	9
<code>a.std()</code>	độ lệch chuẩn	2.571
<code>np.median(a)</code>	trung vị	3.5
<code>np.sort(a)</code>	sắp xếp tăng dần	[1 1 2 3 4 5 6 9]
<code>a.cumsum()</code>	tổng tích lũy	[3 4 8 9 14 23 25 31]

- Tất cả đều chạy trên **cả mảng**, không cần vòng lặp

axis — trục nào bị "gộp lại"?



Lệnh	Ý nghĩa	Kết quả
<code>a.sum()</code>	tổng tất cả	78
<code>a.sum(axis=0)</code>	gộp các hàng → tổng mỗi cột	<code>[15 18 21 24]</code>
<code>a.sum(axis=1)</code>	gộp các cột → tổng mỗi hàng	<code>[10 26 42]</code>

- Mảng trên có `shape = (3, 4)`: `ndim = 2`, trục 0 kích thước 3, trục 1 kích thước 4

Vận dụng: chọn axis nào?

```
# 4 students (rows) x 3 subjects (columns)
diem = np.array([[8.0, 7.0, 9.0],
                 [6.0, 8.5, 7.5],
                 [9.0, 9.0, 8.0],
                 [5.0, 6.0, 6.5]])
```

Viết một dòng lệnh cho mỗi yêu cầu:

- 1 Điểm trung bình của **từng học sinh** (ra 4 số)

Vận dụng: chọn axis nào?

```
# 4 students (rows) x 3 subjects (columns)
diem = np.array([[8.0, 7.0, 9.0],
                 [6.0, 8.5, 7.5],
                 [9.0, 9.0, 8.0],
                 [5.0, 6.0, 6.5]])
```

Viết một dòng lệnh cho mỗi yêu cầu:

- 1 Điểm trung bình của **từng học sinh** (ra 4 số)
→ `diem.mean(axis=1)` `[8. 7.33 8.67 5.83]`

Vận dụng: chọn axis nào?

```
# 4 students (rows) x 3 subjects (columns)
diem = np.array([[8.0, 7.0, 9.0],
                 [6.0, 8.5, 7.5],
                 [9.0, 9.0, 8.0],
                 [5.0, 6.0, 6.5]])
```

Viết một dòng lệnh cho mỗi yêu cầu:

- 1 Điểm trung bình của **từng học sinh** (ra 4 số)
→ `diem.mean(axis=1)` `[8. 7.33 8.67 5.83]`
- 2 Điểm trung bình của **từng môn** (ra 3 số)

Vận dụng: chọn axis nào?

```
# 4 students (rows) x 3 subjects (columns)
diem = np.array([[8.0, 7.0, 9.0],
                 [6.0, 8.5, 7.5],
                 [9.0, 9.0, 8.0],
                 [5.0, 6.0, 6.5]])
```

Viết một dòng lệnh cho mỗi yêu cầu:

- 1 Điểm trung bình của **từng học sinh** (ra 4 số)
→ `diem.mean(axis=1)` `[8. 7.33 8.67 5.83]`
- 2 Điểm trung bình của **từng môn** (ra 3 số)
→ `diem.mean(axis=0)` `[7. 7.625 7.75]`

Vận dụng: chọn axis nào?

```
# 4 students (rows) x 3 subjects (columns)
diem = np.array([[8.0, 7.0, 9.0],
                 [6.0, 8.5, 7.5],
                 [9.0, 9.0, 8.0],
                 [5.0, 6.0, 6.5]])
```

Viết một dòng lệnh cho mỗi yêu cầu:

- 1 Điểm trung bình của **từng học sinh** (ra 4 số)
→ `diem.mean(axis=1)` `[8. 7.33 8.67 5.83]`
- 2 Điểm trung bình của **từng môn** (ra 3 số)
→ `diem.mean(axis=0)` `[7. 7.625 7.75]`
- 3 Điểm **cao nhất** mà mỗi học sinh đạt được

Vận dụng: chọn axis nào?

```
# 4 students (rows) x 3 subjects (columns)
diem = np.array([[8.0, 7.0, 9.0],
                  [6.0, 8.5, 7.5],
                  [9.0, 9.0, 8.0],
                  [5.0, 6.0, 6.5]])
```

Viết một dòng lệnh cho mỗi yêu cầu:

- 1 Điểm trung bình của **từng học sinh** (ra 4 số)
→ `diem.mean(axis=1)` `[8. 7.33 8.67 5.83]`
- 2 Điểm trung bình của **từng môn** (ra 3 số)
→ `diem.mean(axis=0)` `[7. 7.625 7.75]`
- 3 Điểm **cao nhất** mà mỗi học sinh đạt được
→ `diem.max(axis=1)` `[9. 8.5 9. 6.5]`

Vận dụng: chọn axis nào?

```
# 4 students (rows) x 3 subjects (columns)
diem = np.array([[8.0, 7.0, 9.0],
                  [6.0, 8.5, 7.5],
                  [9.0, 9.0, 8.0],
                  [5.0, 6.0, 6.5]])
```

Viết một dòng lệnh cho mỗi yêu cầu:

- 1 Điểm trung bình của **từng học sinh** (ra 4 số)
→ `diem.mean(axis=1)` `[8. 7.33 8.67 5.83]`
- 2 Điểm trung bình của **từng môn** (ra 3 số)
→ `diem.mean(axis=0)` `[7. 7.625 7.75]`
- 3 Điểm **cao nhất** mà mỗi học sinh đạt được
→ `diem.max(axis=1)` `[9. 8.5 9. 6.5]`
- 4 Mỗi học sinh có **mấy môn** từ 8 điểm trở lên?

Vận dụng: chọn axis nào?

```
# 4 students (rows) x 3 subjects (columns)
diem = np.array([[8.0, 7.0, 9.0],
                 [6.0, 8.5, 7.5],
                 [9.0, 9.0, 8.0],
                 [5.0, 6.0, 6.5]])
```

Viết một dòng lệnh cho mỗi yêu cầu:

- 1 Điểm trung bình của **từng học sinh** (ra 4 số)
→ `diem.mean(axis=1)` `[8. 7.33 8.67 5.83]`
- 2 Điểm trung bình của **từng môn** (ra 3 số)
→ `diem.mean(axis=0)` `[7. 7.625 7.75]`
- 3 Điểm **cao nhất** mà mỗi học sinh đạt được
→ `diem.max(axis=1)` `[9. 8.5 9. 6.5]`
- 4 Mỗi học sinh có **mấy môn** từ 8 điểm trở lên?
→ `(diem >= 8).sum(axis=1)` `[2 1 3 0]`

Giữ nguyên số chiều với keepdims

```
tb = diem.mean(axis=1)           # shape (4,)
tb = diem.mean(axis=1, keepdims=True) # shape (4, 1)

print(diem - tb)    # deviation of each score from that
                   # student's own average
```

- `keepdims=True` giữ lại chiều vừa bị gộp: shape `(4,1)` thay vì `(4,)`
- Nhờ vậy `diem - tb` broadcasting đúng: mỗi hàng trừ đi trung bình của chính nó

reshape — thay đổi hình dạng

```
a = np.arange(12)           # [0 1 2 ... 11]   shape (12,)
b = a.reshape(3, 4)         # 3 rows, 4 columns
c = a.reshape(4, 3)         # 4 rows, 3 columns
d = a.reshape(2, 2, 3)      # a 3-D array
e = a.reshape(3, -1)        # -1 = "you figure it out" -> (3,4)

print(b.ravel())            # 1-D; a view when possible, else a
    copy
print(b.flatten())          # 1-D; ALWAYS returns a copy
```

- Số phần tử phải **không đổi**: $3 \times 4 = 12$ (được), còn $5 \times 3 = 15$ thì không
- -1 nghĩa là "chiều này để NumPy tự tính": `a.reshape(3, -1)`
→ NumPy suy ra 4 cột

Thêm chiều cho mảng

```
v = np.array([1, 2, 3])      # shape (3,) - neither row nor col

v[:, np.newaxis]           # shape (3, 1) - COLUMN vector
v[np.newaxis, :]          # shape (1, 3) - ROW vector
v.reshape(-1, 1)           # another way to get a column vector

a = np.array([[1, 2, 3],
              [4, 5, 6]])   # shape (2, 3)
a.T                        # transpose -> shape (3, 2)
```

Lưu ý

Mảng shape (3,) khác shape (3,1) và khác shape (1,3) — phần lớn lỗi broadcasting đến từ chỗ này.

Ghép và tách mảng

```
a = np.array([[1, 2], [3, 4]])
b = np.array([[5, 6], [7, 8]])

np.concatenate([a, b], axis=0)    # stack ROWS    -> (4, 2)
np.concatenate([a, b], axis=1)    # stack COLUMNS -> (2, 4)
np.vstack([a, b])                  # vertical      = axis 0
np.hstack([a, b])                  # horizontal  = axis 1
np.stack([a, b])                   # NEW dimension -> (2, 2, 2)

x = np.arange(10)
np.split(x, 2)                     # split into 2 equal parts
np.split(x, [3, 7])                # split at index 3 and 7
```

- `np.stack` thêm một chiều mới — dùng khi gom nhiều ảnh cùng kích thước thành một chồng ảnh

Tích vô hướng của hai vector

Hai vector **cùng độ dài**: nhân từng cặp phần tử tương ứng rồi cộng tất cả lại.

$$\vec{u} \cdot \vec{v} = u_1 v_1 + u_2 v_2 + u_3 v_3 = 1 \times 4 + 2 \times 5 + 3 \times 6 = 32$$

```
u = np.array([1, 2, 3])
v = np.array([4, 5, 6])

np.dot(u, v)           # 32
(u * v).sum()          # 32 - multiply pairs, then sum
np.linalg.norm(u)      # 3.74 - length of vector u
```

- norm là **độ dài** vector: $\sqrt{u_1^2 + u_2^2 + u_3^2} = \sqrt{14} \approx 3.74$

Nhân ma trận được thực hiện như thế nào?

Với $C = A \times B$:

$$C[i,j] = A[i,:] \cdot B[:,j] = \sum_k A[i,k] B[k,j]$$

hàng i của A nhân vô hướng với cột j của B

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \times \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix} = \begin{pmatrix} 19 & 22 \\ 43 & 50 \end{pmatrix}$$

$$19 = 1 \times 5 + 2 \times 7$$

$$22 = 1 \times 6 + 2 \times 8$$

$$43 = 3 \times 5 + 4 \times 7$$

$$50 = 3 \times 6 + 4 \times 8$$

Điều kiện về hình dạng

$(m, n) @ (n, p) \rightarrow (m, p)$: số **cột** của ma trận trái phải bằng số **hàng** của ma trận phải.

Trong NumPy: * và @ là hai phép khác nhau

```
A = np.array([[1, 2],  
              [3, 4]])  
B = np.array([[5, 6],  
              [7, 8]])  
  
A * B      # [[ 5 12], [21 32]]  ELEMENT-WISE product  
A @ B      # [[19 22], [43 50]]  MATRIX product  
np.dot(A, B)      # same as A @ B  
np.matmul(A, B)   # same as A @ B
```

Lưu ý

* nhân **từng phần tử** (có áp dụng broadcasting). @ là phép **nhân ma trận** trong toán học. Hai kết quả hoàn toàn khác nhau.

Chuyển vị: đổi hàng thành cột

Chuyển vị (transpose): lật ma trận qua **đường chéo chính** — tập hợp các phần tử có chỉ số hàng bằng chỉ số cột ($A[0,0], A[1,1], \dots$). Hàng thứ i trở thành cột thứ i .

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}_{2 \times 3} \quad A^T = \begin{pmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{pmatrix}_{3 \times 2}$$

- Phần tử ở vị trí (hàng i , cột j) chuyển sang vị trí (hàng j , cột i)
- Hình dạng (m, n) trở thành (n, m)
- Trong NumPy chỉ cần viết **A.T** — và đây là một **view**, không tốn thêm bộ nhớ
- Khi hai mảng lệch chiều: $(2,3) @ (2,3)$ không hợp lệ, nhưng $A @ B.T$ thì được

Vì sao hệ phương trình viết được dưới dạng ma trận?

$$\begin{cases} 2x + y = 5 \\ x + 3y = 10 \end{cases} \iff \underbrace{\begin{pmatrix} 2 & 1 \\ 1 & 3 \end{pmatrix}}_A \underbrace{\begin{pmatrix} x \\ y \end{pmatrix}}_{\vec{x}} = \underbrace{\begin{pmatrix} 5 \\ 10 \end{pmatrix}}_{\vec{b}}$$

Nhân **hàng 1** của A với vector ẩn theo đúng quy tắc ở slide trước:
 $2 \times x + 1 \times y$ — chính là vế trái của phương trình thứ nhất. Tương tự hàng 2 cho ra $x + 3y$.

```
A = np.array([[2, 1], [1, 3]])  
b = np.array([5, 10])  
  
np.linalg.solve(A, b)      # [1. 3.]  -> x = 1, y = 3  
A @ np.array([1, 3])      # [ 5 10]  check: correct
```

Ảnh số chính là một mảng NumPy

```
# color image: (height, width, 3 RGB channels), uint8 0..255
img = rng.integers(0, 256, size=(128, 128, 3), dtype=np.uint8)

img.shape          # (128, 128, 3)
img[0, 0]          # [R G B] of the top-left pixel

img_gray = img.mean(axis=2)          # grayscale: mean of RGB
img_flip = img[:, ::-1, :]          # flip left-right
img_crop = img[32:96, 32:96]          # crop the centre
img_norm = img.astype(np.float32) / 255.0 # scale to [0, 1]

do_sang = img.mean()                # average brightness
kenh_do = img[:, :, 0]              # the red channel alone
```

- Lật ảnh, cắt ảnh, đổi độ sáng — các thao tác xử lý ảnh quen thuộc — chỉ là slicing và phép toán trên mảng

softmax — biến dãy số thành tỉ lệ phần trăm

$$\text{softmax}(x)_i = \frac{e^{x_i}}{e^{x_1} + e^{x_2} + \dots + e^{x_n}}$$

```
def softmax(x):  
    e = np.exp(x - np.max(x))    # subtract max: avoid overflow  
    return e / np.sum(e)  
  
diem_so = np.array([2.0, 1.0, 0.1])  
print(softmax(diem_so))          # [0.659 0.242 0.099] - sums 1
```

- Mọi kết quả đều **dương** và **tổng bằng 1** → đọc được như tỉ lệ phần trăm: 65.9%, 24.2%, 9.9%
- Số nào lớn hơn thì tỉ lệ cao hơn — nhưng chênh lệch bị khuếch đại (do hàm e^x)

Đo sai số giữa hai dãy số

Ví dụ: nhiệt độ *dự báo* và nhiệt độ *thực tế* của 4 ngày — dự báo lệch trung bình bao nhiêu độ?

```
du_bao    = np.array([28.0, 31.0, 27.5, 33.0])
thuc_te   = np.array([27.8, 31.2, 29.0, 32.5])

sai_so = du_bao - thuc_te           # [0.2 -0.2 -1.5 0.5]

# mean absolute error (MAE)
np.mean(np.abs(sai_so))             # 0.6 degrees

# mean squared error (MSE) - penalises big errors more
np.mean(sai_so ** 2)                # 0.645
```

Đếm tỉ lệ giống nhau giữa hai dãy:

```
dap_an    = np.array([1, 3, 2, 4, 1])   # correct answers
bai_lam    = np.array([1, 3, 4, 4, 2])   # student's answers
np.mean(dap_an == bai_lam)              # 0.6 -> 60% correct
```

Bảng tra cứu nhanh

Việc cần làm	Lệnh
Tạo mảng từ list	<code>np.array([1,2,3])</code>
Mảng số 0 / số 1	<code>np.zeros((m,n)), np.ones((m,n))</code>
Dãy số	<code>np.arange(start,stop,step)</code> <code>np.linspace(start,stop,num)</code>
Số ngẫu nhiên	<code>rng = np.random.default_rng(42)</code>
Xem hình dạng	<code>a.shape, a.ndim, a.dtype</code>
Đổi kiểu	<code>a.astype(np.float32)</code>
Lấy hàng / cột	<code>a[i], a[:, j]</code>
Lọc theo điều kiện	<code>a[a > 5]</code>
if-else cho mảng	<code>np.where(cond, x, y)</code>
Sao chép thật sự	<code>a.copy()</code>
Thống kê	<code>a.sum(), a.mean(), a.std()</code>
Theo hàng / cột	<code>a.sum(axis=1), a.sum(axis=0)</code>
Đổi hình dạng	<code>a.reshape(m, -1)</code>
Chuyển vị	<code>a.T</code>
Ghép mảng	<code>np.vstack, np.hstack, np.concatenate</code>
Nhân ma trận	<code>A @ B</code>
Tích vô hướng	<code>np.dot(u, v)</code>
Khoảng cách / độ dài	<code>np.linalg.norm(a - b)</code>
Giải hệ phương trình	<code>np.linalg.solve(A, b)</code>
Sắp xếp	<code>np.sort(a), np.argsort(a)</code>

Bản đầy đủ hơn (tiếng Anh): [geeksforgeeks.org/python/numpy-cheat-sheet](https://www.geeksforgeeks.org/python/numpy-cheat-sheet)

Bài tập tại lớp

```
rng = np.random.default_rng(42)

diem = rng.integers(0, 11, 10)           # exercise 3
bang_diem = rng.uniform(4, 10, (5, 3))  # exercise 4
x = rng.normal(50, 15, 20)              # exercise 5
```

- 1 Tạo mảng các số chẵn từ 0 đến 20; in tổng và trung bình cộng
- 2 Tạo ma trận 3×3 chứa các số 1–9; in hàng thứ hai, cột cuối, đường chéo chính
- 3 Với `diem`: đếm số học sinh đạt (≥ 5) và in ra điểm của những em đó
- 4 Với `bang_diem` (5 học sinh $\times$ 3 môn): tính điểm trung bình mỗi học sinh, mỗi môn, và tìm học sinh có trung bình cao nhất
- 5 Với `x`: chuẩn hóa về khoảng $[0, 1]$ theo công thức $x' = \frac{x - \min}{\max - \min}$

Bài tập về nhà

```
rng = np.random.default_rng(0)
du_bao = rng.uniform(25, 35, 7)
thuc_te = du_bao + rng.normal(0, 1, 7)
X = rng.normal(50, 10, (100, 5))
img = rng.integers(0, 256, (64, 64, 3), dtype=np.uint8)
A, B = rng.integers(0, 5, (3, 2)), rng.integers(0, 5, (2, 4))
```

- 1 Tính MAE và MSE giữa `du_bao` và `thuc_te`, không dùng vòng lặp
- 2 Chuẩn hóa z-score từng cột của `X`; kiểm tra trung bình ≈ 0 , độ lệch chuẩn ≈ 1
- 3 Với `img`: chuyển sang ảnh xám, lật ngang, cắt vùng giữa 32×32
- 4 Tính `A @ B`; tự kiểm chứng một phần tử bằng `np.dot`
- 5 Viết hệ $3x + 2y = 12$, $x - y = 1$ dưới dạng ma trận và giải bằng `np.linalg.solve`
- 6 **Nâng cao:** viết hàm `softmax(x)`, kiểm tra tổng kết quả bằng 1

Tóm tắt buổi học

- **ndarray** — mảng nhiều chiều, phần tử cùng **dtype**, được tổ chức và xử lý hiệu quả trong bộ nhớ
- **Vector hóa** — tính trên cả mảng, không cần vòng lặp
- **Broadcasting** — tự khớp hình dạng giữa các mảng khác kích thước
- **shape** và **axis** — hai khái niệm phải nắm chắc
- **View** và **copy** — slicing dùng chung bộ nhớ
- **@** — nhân ma trận: hàng nhân cột rồi cộng lại

Buổi sau

Pandas — xử lý dữ liệu dạng bảng, và **Matplotlib** — trực quan hóa dữ liệu.

Tài liệu: numpy.org/doc/stable/user/absolute_beginners.html
Cheat sheet: [geeksforgeeks.org/python/numpy-cheat-sheet](https://www.geeksforgeeks.org/python/numpy-cheat-sheet)